

PENGEMBANGAN APLIKASI SIMULASI MANAJEMEN DIREKTORI FOLDER LAPTOP BERBASIS CLI MENGGUNAKAN PYTHON

Disusun untuk memenuhi tugas akhir Project-Based Learning (PBL)

Mata Kuliah Algoritma & Struktur Data (TPL2106)

Disusun oleh:

NAMA	NIM
Gabriella Anggi Apriliana Tambun	J0403251072
Mouzia Syawalova Merro	J0403251080
Alyaa Mardlatillah Sofwan	J0403251148



Program Studi Teknologi Rekayasa Perangkat Lunak

Sekolah Vokasi IPB University

2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, laporan akhir *Project-Based Learning* (PBL) berjudul "Pengembangan Aplikasi Simulasi Manajemen Direktori Folder Laptop Berbasis CLI Menggunakan Python" ini dapat diselesaikan dengan baik. Laporan ini disusun untuk memenuhi tugas akhir mata kuliah Algoritma & Struktur Data (TPL2106) di Program Studi Teknologi Rekayasa Perangkat Lunak, Sekolah Vokasi IPB University. Melalui proyek ini, penulis berhasil mengimplementasikan berbagai jenis struktur data ke dalam sebuah solusi nyata berupa simulasi sistem manajemen berkas komputer. Penulis mengucapkan terima kasih yang sebesar-besarnya kepada Ibu Dr. Shelve Nidya Neyman dan Ibu Dr. Sofiyanti Indriasari S.Kom, M.Kom selaku dosen pengampu yang telah memberikan bimbingan, ilmu, dan arahan berharga selama pengerjaan proyek ini, serta kepada rekan kelompok dan orang tua atas segala dukungannya.

Penulis menyadari bahwa laporan ini masih jauh dari kesempurnaan, baik dari segi penyusunan maupun materi teknis di dalamnya. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa yang akan datang. Akhir kata, semoga laporan proyek PBL ini dapat memberikan manfaat, wawasan baru, serta kontribusi positif bagi perkembangan ilmu pengetahuan, khususnya di bidang pemrograman dan struktur data.

DAFTAR ISI

KATA PENGANTAR.....	2
DAFTAR ISI.....	3
BAB I. PENDAHULUAN.....	5
1.1 Latar Belakang.....	5
1.2 Tujuan Proyek.....	5
1.3 Ruang Lingkup.....	6
BAB II. ANALISIS DAN PERANCANGAN.....	7
2.1 Deskripsi Sistem.....	7
2.2 Struktur Data yang Digunakan.....	7
2.3 Algoritma yang Digunakan.....	8
2.4 Diagram Alir Program (Flowchart).....	9
2.5 Struktur Program.....	9
BAB III. IMPLEMENTASI PROGRAM.....	11
3.1 Tampilan Menu Utama.....	11
3.2 Fitur Create.....	11
3.3 Fitur Read.....	12
3.4 Fitur Update.....	13
3.5 Fitur Delete.....	14
3.6 Fitur Searching.....	15
3.7 Fitur Sorting.....	15
3.8 Penyimpanan Data (File Handling).....	16
3.9 Validasi Input dan Error Handling.....	17
BAB IV. PENGUJIAN DAN ANALISIS.....	19
4.1 Skenario Pengujian.....	19
4.2 Kelebihan Sistem.....	20
4.3 Keterbatasan Sistem.....	21

BAB V. KENDALA DAN SOLUSI.....	22
BAB VI. PEMBAGIAN TUGAS ANGGOTA.....	23
BAB VII. KESIMPULAN DAN SARAN.....	24
Kesimpulan.....	24
Saran Pengembangan.....	24
DAFTAR PUSTAKA.....	25
LAMPIRAN.....	26
Lampiran A. Link GitHub.....	26
Lampiran B. Screenshot Program.....	26
Lampiran C. Struktur Folder Proyek.....	26
Lampiran D. Commit History GitHub.....	27
Lampiran E. Source Code Utama.....	27

BAB I. PENDAHULUAN

1.1 Latar Belakang

Dalam sistem operasi komputer, pengelolaan berkas dan folder merupakan salah satu contoh nyata dari penerapan struktur data non-linear di kehidupan sehari-hari. Setiap hari pengguna berinteraksi dengan ribuan dokumen yang tersusun di dalam direktori laptop, namun mekanisme di balik layar mengenai bagaimana sistem mengelola hubungan antar-folder tersebut jarang dipahami secara mendalam. Berangkat dari topik "Direktori Folder Laptop" yang diperoleh, kelompok kami melihat peluang besar untuk mengeksplorasi secara langsung bagaimana data hierarkis diorganisasikan, dilacak, dan dimanipulasi melalui pendekatan algoritma yang efisien.

Proyek *Project-Based Learning* (PBL) ini dirancang untuk mensimulasikan cara kerja *File Explorer* sederhana berbasis Command Line Interface (CLI) menggunakan bahasa pemrograman Python. Dalam pengembangannya, aplikasi ini mengimplementasikan struktur data *General Tree* (Pohon N-ary) untuk merepresentasikan struktur folder bertingkat, di mana sebuah folder bertindak sebagai *parent node* yang dapat memiliki banyak subfolder sebagai *children nodes*. Selain itu, struktur data *Stack* juga diterapkan guna mengelola riwayat navigasi jalur (*path tracking*), sehingga pengguna dapat berpindah direktori ke atas atau kembali ke folder sebelumnya dengan prinsip *Last In First Out* (LIFO).

Tantangan utama dalam simulasi ini tidak hanya terletak pada visualisasi hirarki data di terminal, melainkan juga pada aspek persistensi data. Tanpa adanya mekanisme penyimpanan yang baik, seluruh struktur direktori yang telah dibuat oleh pengguna akan hilang begitu program dimatikan. Oleh karena itu, proyek ini turut mengintegrasikan konsep *file handling* agar seluruh perubahan struktur pohon folder dapat disimpan secara permanen ke dalam file eksternal. Melalui integrasi struktur data dan *file handling* ini, aplikasi diharapkan dapat menjadi media pembelajaran yang konkret dalam memahami arsitektur manajemen berkas pada sistem operasi.

1.2 Tujuan Proyek

1. Membuat aplikasi simulasi Direktori Folder Laptop berbasis terminal (Command Line Interface) menggunakan bahasa pemrograman Python.
2. Mengimplementasikan struktur data General Tree untuk merepresentasikan hierarki folder secara bertingkat secara akurat.
3. Mengimplementasikan struktur data Stack untuk mengelola fitur riwayat (history) navigasi jalur direktori berdasarkan prinsip LIFO.
4. Menerapkan algoritma Sorting untuk mengurutkan dan merapikan tampilan daftar file atau subfolder berdasarkan alfabet.
5. Menerapkan algoritma Searching (DFS) untuk melakukan pencarian mendalam terhadap file atau folder di dalam direktori.
6. Menerapkan konsep file handling dengan format .csv untuk menjaga persistensi data agar struktur direktori tersimpan secara permanen.

1.3 Ruang Lingkup

- Platform CLI (Console)
- Bahasa Python
- Penyimpanan file CSV
- Fitur CRUD
- Searching Sorting

BAB II. ANALISIS DAN PERANCANGAN

2.1 Deskripsi Sistem

Sistem "Direktori Folder Laptop" merupakan sebuah program simulasi berbasis antarmuka baris perintah (*Command Line Interface* atau CLI) yang dirancang untuk merepresentasikan manajemen berkas dan direktori pada suatu sistem operasi. Tujuan utama dari pengembangan sistem ini adalah memberikan pemahaman mendalam mengenai penerapan struktur data hierarkis non-linear dalam mengorganisasikan data yang memiliki hubungan induk-anak (*parent-child*). Melalui sistem ini, pengguna dapat mensimulasikan aktivitas pengelolaan ruang penyimpanan lokal komputer secara virtual tanpa melakukan modifikasi langsung pada sistem berkas fisik perangkat keras.

Sistem ini bekerja dengan memanfaatkan interaksi menu interaktif yang menyediakan fungsionalitas pengelolaan data secara dinamis. Saat program dijalankan, sistem akan mencoba memulihkan status (*state*) struktur direktori dari sesi sebelumnya melalui pembacaan basis data teks. Pengguna kemudian disajikan informasi mengenai lokasi jalur (*path*) aktif saat ini secara *real-time* dan dapat mengeksekusi berbagai instruksi melalui kode angka dari 0 sampai 8. Setiap perubahan struktur data yang disebabkan oleh interaksi pengguna akan secara otomatis diperbarui di dalam memori dan disinkronisasikan ke dalam media penyimpanan eksternal untuk menjamin konsistensi data.

Secara fungsional, sistem menyediakan fitur *Create*, *Read*, *Update*, dan *Delete* (CRUD) yang komprehensif, di mana pengguna dapat membuat folder atau file baru dengan mekanisme validasi anti-duplikasi, melihat isi direktori berdasarkan filter tipe tertentu, mengubah penamaan entitas, serta menghapus sub-folder atau file secara instan. Selain operasi dasar tersebut, sistem ini dilengkapi dengan fitur tingkat lanjut berupa navigasi folder (masuk ke sub-folder dan kembali ke direktori utama), pengurutan isi direktori secara alfabetis (A-Z), serta pencarian data secara menyeluruh dan mendalam di seluruh tingkat hierarki.

2.2 Struktur Data yang Digunakan

Program Direktori Folder Laptop memanfaatkan beberapa struktur data untuk mendukung pengelolaan folder dan file secara efisien. Struktur data yang digunakan dapat dilihat pada Tabel.

Struktur Data	Fungsi
Class Node	Merepresentasikan entitas tunggal berupa berkas (File) atau direktori (Folder) yang menyimpan atribut nama, tipe, penunjuk induk, dan kontainer internal.
Tree	Menyimpan keseluruhan hierarki direktori secara non-linear, menghubungkan simpul induk (<i>parent</i>) dengan simpul anak (<i>child</i>).
List	Menyimpan daftar objek turunan di dalam variabel <code>self.isi</code> pada setiap folder secara dinamis.
Stack	Menyimpan data riwayat penelusuran folder (<code>stack_riwayat</code>) guna memfasilitasi navigasi kembali (<i>backtracking</i>).
Dictionary	Memetakan nama folder ke objek Node (<code>nodes</code>) saat proses pembacaan file CSV untuk mempermudah pencarian referensi <i>parent</i> .
File CSV	Berperan sebagai media penyimpanan eksternal yang bersifat permanen (<i>data persistence</i>) untuk mencatat seluruh struktur pohon.

Alasan Pemilihan Struktur data

- *Class Node* dipilih sebagai representasi objek dasar karena program ini mengadopsi paradigma Pemrograman Berorientasi Objek (*Object-Oriented Programming*). Melalui *Class Node*, konsep enkapsulasi dapat diterapkan untuk membungkus atribut-atribut esensial berkas (seperti nama, tipe, kontainer isi, dan pointer lokasi) ke dalam satu kesatuan entitas yang modular, sehingga mempermudah manipulasi data secara dinamis.
- Struktur data Tree dipilih karena memiliki karakteristik hubungan hierarkis bercabang yang paling akurat dalam merepresentasikan sistem berkas (*file system*) pada sistem operasi komputer di dunia nyata. Melalui Tree, aturan di mana satu folder dapat memiliki banyak sub-folder/file tetapi setiap anak hanya boleh memiliki satu induk tunggal dapat terpenuhi secara logis.
- Struktur data List dipilih untuk mengisi atribut *self.isi* karena sifatnya yang dinamis (*dynamic array*). Hal ini mempermudah proses penambahan (*append*) dan penghapusan (*remove*) berkas atau folder di dalam direktori aktif tanpa perlu mendeklarasikan alokasi ukuran memori statis sejak awal program.
- Stack dipilih untuk mengelola navigasi folder karena mengikuti prinsip LIFO (*Last-In, First-Out*). Ketika pengguna masuk lebih dalam ke sebuah sub-folder, folder sebelumnya akan ditumpuk (*push*). Saat pengguna memilih opsi "Kembali", folder terakhir yang dimasukkan adalah yang pertama kali dikeluarkan (*pop*), sehingga proses pelacakan jalur kembali (*backtracking*) berjalan runtut.
- Dictionary digunakan pada fase inisialisasi pembacaan CSV karena memanfaatkan mekanisme *hashing* dengan kompleksitas waktu rata-rata $O(1)$. Hal ini membuat pencarian dan pengaitkan objek anak ke objek induknya berdasarkan kunci nama (*string*) dapat dilakukan secara instan tanpa perlu melakukan perulangan sekuensial yang memakan waktu.
- File CSV dipilih sebagai media penyimpanan permanen karena formatnya yang sederhana, berbasis teks standar, dan memiliki ukuran penyimpanan yang sangat ringan. CSV sangat efisien untuk menyimpan data relasional mentah yang merepresentasikan struktur pohon ([Nama, Tipe, Parent]) tanpa memerlukan *overhead* atau konfigurasi sistem manajemen basis data (DBMS) yang kompleks.

2.3 Algoritma yang Digunakan

Program Direktori Folder Laptop menerapkan beberapa algoritma untuk mendukung proses pengelolaan data, yaitu algoritma pencarian (searching), pengurutan (sorting), traversal tree, dan operasi CRUD.

- Searching (Sequential Search Rekursif)

Proses pencarian file dan folder dilakukan menggunakan algoritma Sequential Search yang dikombinasikan dengan teknik rekursif pada struktur tree. Algoritma akan menelusuri setiap node mulai dari root hingga seluruh subfolder untuk mencari nama yang sesuai dengan kata kunci yang dimasukkan pengguna.

Alasan Penggunaan:

1. Struktur data yang digunakan berupa tree sehingga data tidak tersusun secara teratur.
2. Sequential Search mudah diimplementasikan pada traversal tree.
3. Cocok untuk jumlah data yang tidak terlalu besar.

- Sorting (Built-in Sort / Timsort)

Proses pengurutan isi folder dilakukan menggunakan fungsi `sort()` bawaan Python dengan parameter `key=lambda x: x.folder.lower()`. Secara internal, Python menggunakan algoritma Timsort, yaitu kombinasi antara Merge Sort dan Insertion Sort.

Alasan Penggunaan:

1. Memiliki performa yang baik pada berbagai kondisi data.
2. Implementasi lebih sederhana dibanding membuat algoritma sorting secara manual.
3. Mendukung pengurutan data secara alfabetis tanpa membedakan huruf besar dan kecil.

- Traversal Tree

Traversal tree digunakan pada proses pencarian data dan penyimpanan data ke file CSV. Algoritma akan mengunjungi setiap node dalam struktur pohon secara rekursif mulai dari root hingga seluruh child node selesai diproses.

Alasan Penggunaan:

1. Mampu menjangkau seluruh folder dan file dalam sistem direktori.
2. Sesuai dengan struktur data tree yang digunakan.
3. Mempermudah proses pencarian dan penyimpanan data secara menyeluruh.

- CRUD (Create, Read, Update, Delete)

Program menerapkan algoritma CRUD sebagai operasi utama pengelolaan data.

1. Create: Menambahkan folder atau file baru ke dalam direktori.
2. Read: Menampilkan isi folder sesuai filter yang dipilih pengguna.
3. Update: Mengubah nama folder atau file yang telah ada.
4. Delete: Menghapus folder atau file dari direktori.

Alasan Penggunaan:

1. CRUD merupakan operasi dasar yang wajib tersedia pada sistem manajemen data.
2. Memungkinkan pengguna melakukan pengelolaan direktori secara lengkap.
3. Mendukung kebutuhan utama sistem dalam memanipulasi folder dan file.

- Algoritma Penyimpanan dan Pembacaan CSV

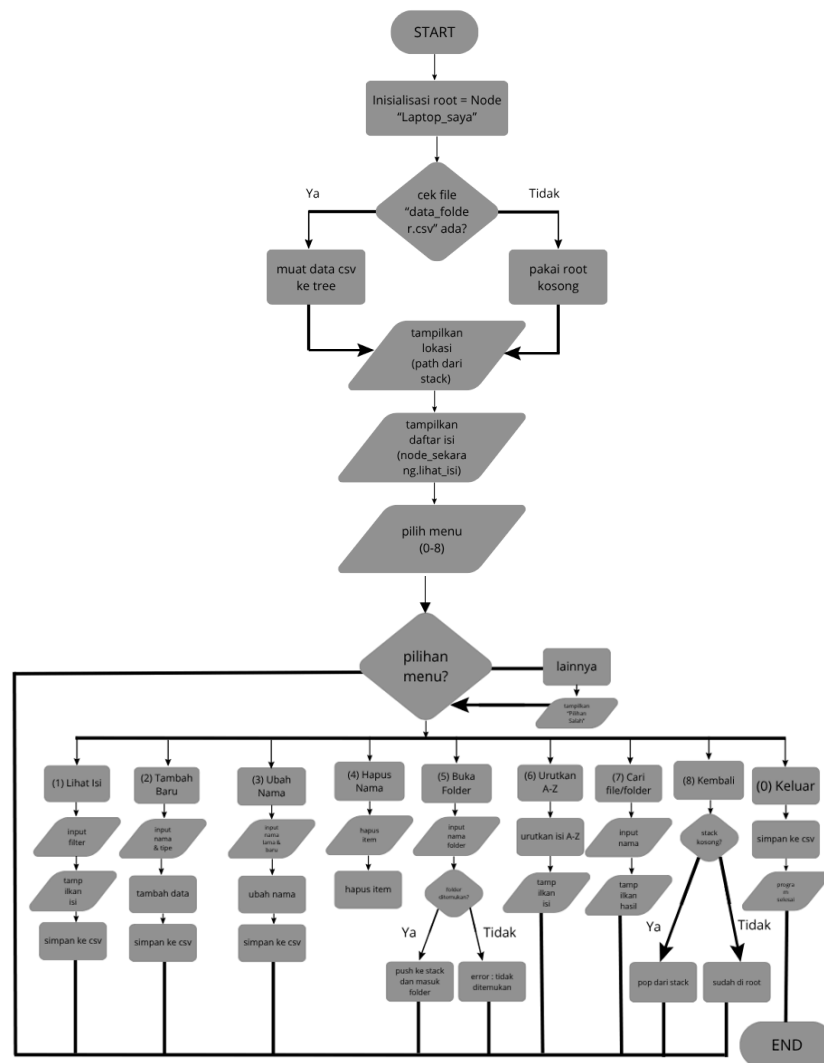
Algoritma ini digunakan untuk menyimpan dan memuat data direktori secara permanen. Saat penyimpanan, sistem melakukan traversal tree untuk menulis seluruh node ke dalam file

CSV. Saat pembacaan, sistem membaca setiap baris CSV dan membangun kembali struktur direktori sesuai hubungan parent-child yang tersimpan.

Alasan Penggunaan:

1. Format CSV sederhana dan mudah diproses menggunakan Python.
2. Tidak memerlukan database tambahan.
3. Cocok untuk aplikasi skala kecil dan menengah.

2.4 Diagram Alir Program (Flowchart)



2.5 Struktur Program

Program "Direktori Folder Laptop" dirancang menggunakan pendekatan arsitektur modular. Kode program tidak menumpuk dalam satu berkas tunggal, melainkan dipecah ke dalam beberapa modul berkas komponen Python (.py) yang terpisah berdasarkan tanggung jawab spesifiknya (*Separation of Concerns*). Pendekatan modular ini bertujuan untuk mempermudah pembagian kerja di dalam tim pengembang, meningkatkan keterbacaan kode (*code readability*), serta mempermudah proses pelacakan kesalahan (*debugging*) maupun pengembangan fitur di masa mendatang.

Adapun spesifikasi dan fungsi dari setiap berkas yang dibuat di dalam sistem ini adalah sebagai berikut:

- main.py

Merupakan berkas program utama yang berfungsi sebagai penggerak alur dan pusat kendali aplikasi (*main control loop*). Di dalam berkas ini terdapat kode untuk mengimpor seluruh modul eksternal, mengelola variabel navigasi global (*node_sekarang* dan *stack_riwayat*), menangani inisialisasi awal saat program dijalankan, serta mengimplementasikan antarmuka interaktif berbasis teks (*Command Line Interface*) untuk menerima input menu dari pengguna (angka 0–8).

- node.py

Berkas ini merupakan representasi dari lapisan pemodelan objek (*object modeling layer*). Di dalamnya didefinisikan cetak biru Class Node beserta metode-metode internal untuk memanipulasi status (*state*) dari simpul yang sedang aktif. Fungsi utama berkas ini adalah menangani operasi dasar CRUD lokal, yang meliputi penambahan data (*tambah_isi*), penampilan isi folder (*lihat_isi*), pembaruan nama objek (*ubah_nama*), dan penghapusan objek berkas (*hapus_isi*).

- file_handler.py

Modul ini bertanggung jawab secara penuh terhadap manajemen penyimpanan data eksternal (*data persistence*). Berkas ini mengimplementasikan fungsi *simpan_ke_csv()* dan fungsi rekursif pembantu *_tulis_rekursif()* yang bertugas mengubah struktur data

pohon (*tree*) yang dinamis di dalam memori komputer menjadi format baris teks terstruktur yang siap ditulis ke dalam media penyimpanan lokal.

- `searching.py`

Berkas ini khusus mengisolasi fungsionalitas pencarian data global di dalam sistem direktori. Di dalamnya diimplementasikan fungsi pembungkus `cari_file_folder()` dan fungsi inti `_cari_rekursif()` yang mengadopsi algoritma penulisan mendalam (*depth-first approach*) untuk menyisir setiap cabang sub-folder tanpa batasan tingkatan demi menemukan objek berkas yang dicari pengguna.

- `sorting.py`

Modul ini mengisolasi logika pengurutan data di dalam kontainer direktori. Di dalam berkas ini didefinisikan fungsi `urutkan_isi()` yang memanfaatkan algoritma hibrida stabil *Timsort* (fungsi `.sort()` bawaan Python) yang dikombinasikan dengan ekspresi lambda untuk menyusun kembali daftar berkas secara alfabetis (A–Z) tanpa bersifat sensitif terhadap huruf kapital (*case-insensitive*).

- `data_folder.csv`

Merupakan berkas penyimpanan sekunder berbasis teks dengan format *Comma-Separated Values* (CSV). Berkas ini bertindak sebagai basis data sederhana yang menyimpan salinan fisik struktur pohon dari sesi aplikasi sebelumnya, menggunakan format kolom relasional berupa tabel tiga elemen, yaitu Nama, Tipe, dan Parent.

Seluruh komponen berkas di atas saling terintegrasi saat program dijalankan. Berkas `main.py` bertindak sebagai integrator yang memanggil entitas dari `node.py`, memanfaatkan fungsionalitas pemrosesan data dari `searching.py` dan `sorting.py`, serta menginstruksikan `file_handler.py` untuk mensinkronisasikan perubahan data ke dalam berkas `data_folder.csv` setiap kali pengguna selesai melakukan manipulasi direktori secara *real-time*.

BAB III. IMPLEMENTASI PROGRAM

3.1 Tampilan Menu Utama

```
=====
Selamat Datang di Direktori Folder Laptop Saya!
Sistem siap mengelola file dan folder Anda.
=====
Tekan Enter untuk masuk ke program...

=====
📁 LOKASI SAAT INI: Laptop_saya
=====
1. Lihat isi folder
2. Tambah folder/file baru
3. Ubah nama
4. Hapus folder/file
5. Buka (Folder)
6. Urutkan (A-Z)
7. Cari nama
8. Kembali
0. Keluar dari program
=====
Pilih menu (0-8): █
```

3.2 Fitur Create

Fitur *Create* pada aplikasi ini diimplementasikan melalui sub-menu "Tambah folder/file baru" yang berfungsi untuk menyisipkan entitas baru, baik berupa folder maupun file, ke dalam direktori aktif saat itu. Secara teknis, ketika pengguna memanggil operasi ini, program akan meminta tiga masukan utama, yaitu nama dari elemen yang akan dibuat dan tipe entitasnya. Angka 1 untuk representasi 'Folder' dan angka 2 untuk 'File'.

Dari aspek struktur data *General Tree*, proses ini bekerja dengan cara menginstansiasi sebuah objek *node* baru yang kemudian disisipkan ke dalam daftar anak (*children list*) milik *node* direktori aktif. Di samping itu, program juga memastikan bahwa nama yang dimasukkan bersifat unik dan belum digunakan oleh objek lain di dalam tingkat direktori yang sama.

Berdasarkan pengujian, ketika pengguna memilih opsi untuk menambah data, memasukkan nama elemen berupa "job", dan memilih tipe "1" (Folder), sistem secara otomatis memproses relasi hierarki tersebut. Begitu proses pengaitan *node* anak ke *node* induk selesai di

dalam memori, program akan memicu fungsi *file handling* untuk langsung memperbarui data pada penyimpanan permanen. Operasi diakhiri dengan menampilkan pesan konfirmasi kontekstual berupa “[Sukses] Berhasil menambahkan Folder ke 'Laptop_saya’”, yang menandakan bahwa folder baru telah sukses terintegrasi ke dalam struktur pohon direktori.

```
Pilih menu (0-8): 2

--- TAMBAH BARU ---
Kosongkan untuk membatalkan.
Masukkan nama: job
Tipe (1. Folder, 2. File): 1
[Sukses] Berhasil menambahkan Folder ke 'Laptop_saya'

Tekan Enter untuk kembali ke menu...
```

```
Pilih menu (0-8): 2

--- TAMBAH BARU ---
Kosongkan untuk membatalkan.
Masukkan nama: kode
Tipe (1. Folder, 2. File): 2
[Sukses] Berhasil menambahkan File ke 'Laptop_saya'

Tekan Enter untuk kembali ke menu...
```

```
=====
Pilih menu (0-8): 2

--- TAMBAH BARU ---
Kosongkan untuk membatalkan.
Masukkan nama: work
Tipe (1. Folder, 2. File): 1
Folder 'work' sudah ada di lokasi ini!
```

3.3 Fitur Read

Fitur Read pada aplikasi simulasi ini dirancang dengan fungsi penyaringan (filtering) agar penelusuran lebih efisien. Fitur ini bekerja dengan membaca daftar objek anak (children list) dari node Tree yang sedang diakses. Ketika fungsi dipanggil, sistem akan memberikan tiga opsi penyaringan kepada pengguna, yaitu menampilkan kelompok folder saja, menampilkan file saja, atau menampilkan semua entitas tanpa pengecualian. Program kemudian melakukan iterasi

terhadap seluruh elemen anak dan menyaringnya berdasarkan atribut tipe data objek sebelum dicetak ke layar terminal.

Berdasarkan implementasi, ketika pengguna memilih menu 1 dan memasukkan angka 3 untuk menampilkan semua data, sistem mengeksekusi fungsi pembacaan secara menyeluruh. Sedangkan, jika pengguna memasukkan angka 1, akan dilakukan filterisasi untuk menampilkan tipe “Folder” saja. Dan ketika pengguna memasukkan angka 2, maka hanya akan ditampilkan tipe “File” saja. Proses penelusuran linear ini terbukti efektif dalam menyajikan informasi hierarki direktori aktif secara real-time kepada pengguna sebelum kembali ke menu utama.

```

=====
Pilih menu (0-8): 1
Tampilkan hanya (1: Folder, 2: File, 3: Semua): 3

--- HASIL FILTER ---
  📁 kerja
  📄 kerjaan
  📁 work
  📄 kampus
  📁 ILY ITS
  📄 kode
  📁 job

Tekan Enter untuk kembali ke menu...

```

```

=====
Pilih menu (0-8): 1
Tampilkan hanya (1: Folder, 2: File, 3: Semua): 1

--- HASIL FILTER ---
(Menampilkan hanya: Folder)
  📁 kerja
  📁 work
  📁 ILY ITS
  📁 job

Tekan Enter untuk kembali ke menu...

```

```

=====
Pilih menu (0-8): 1
Tampilkan hanya (1: Folder, 2: File, 3: Semua): 2

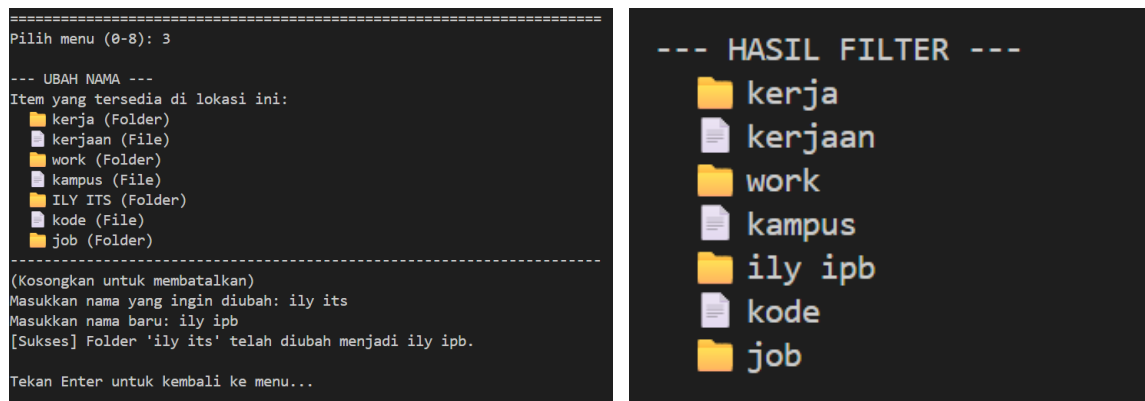
--- HASIL FILTER ---
(Menampilkan hanya: File)
  📄 kerjaan
  📄 kampus
  📄 kode

```

3.4 Fitur Update

Fitur Update berfungsi untuk mengganti nama dari folder atau file yang berada di dalam direktori aktif saat itu. Pada struktur data General Tree, proses pembaruan ini tidak mengubah relasi hierarki ataupun memindahkan posisi node di dalam struktur pohon direktori. Operasi ini hanya memodifikasi atribut string nama pada objek node anak yang ditargetkan. Sebelum melakukan perubahan, program akan melakukan iterasi pencarian linear untuk memastikan bahwa item yang ingin diubah memang ada di lokasi tersebut, sekaligus memvalidasi agar nama baru yang diinput belum digunakan oleh file atau folder lain di dalam direktori yang sama guna menghindari duplikasi data.

Berdasarkan implementasi, ketika pengguna memilih opsi menu 3, sistem terlebih dahulu menampilkan seluruh item yang tersedia di lokasi aktif agar mempermudah pengguna. Ketika pengguna memasukkan nama objek yang ingin diubah yaitu "ily its" dan mengetikkan nama baru berupa "ily ipb", program segera mencari node tersebut, memperbarui datanya di memori, dan mengembalikannya ke dalam sistem penyimpanan file. Proses ini diakhiri dengan pesan konfirmasi keberhasilan sistem: [Sukses] Folder 'ily its' telah diubah menjadi ily ipb., yang membuktikan bahwa atribut objek node dalam Tree telah berhasil diperbarui secara real-time.



```
Pilih menu (0-8): 3
-----
--- UBAH NAMA ---
Item yang tersedia di lokasi ini:
  [Folder] kerja
  [File] kerjaan
  [Folder] work
  [File] kampus
  [Folder] ILY ITS
  [File] kode
  [Folder] job
-----
(Kosongkan untuk membatalkan)
Masukkan nama yang ingin diubah: ily its
Masukkan nama baru: ily ipb
[Sukses] Folder 'ily its' telah diubah menjadi ily ipb.
Tekan Enter untuk kembali ke menu...

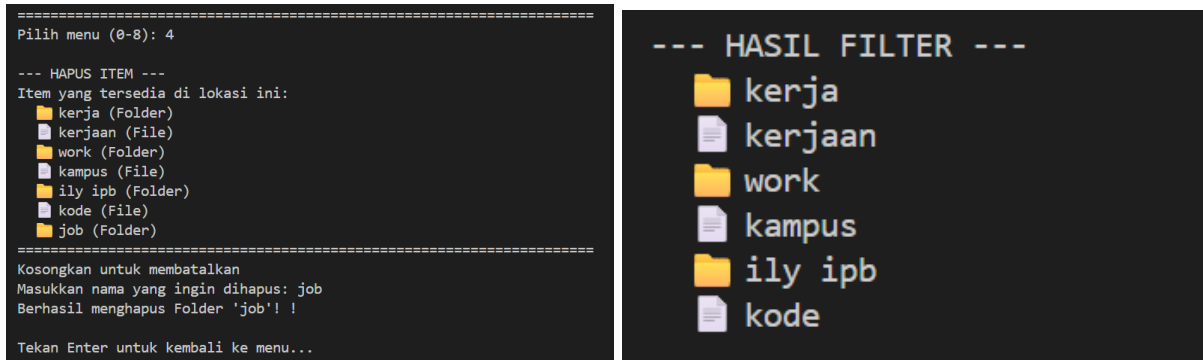
--- HASIL FILTER ---
  [Folder] kerja
  [File] kerjaan
  [Folder] work
  [File] kampus
  [Folder] ily ipb
  [File] kode
  [Folder] job
```

3.5 Fitur Delete

Fitur Delete berfungsi untuk membuang entitas tertentu dari lokasi direktori aktif. Proses penghapusan ini memerlukan ketelitian, terutama jika item yang dihapus berupa folder yang memiliki sub-folder. Ketika perintah eksekusi dijalankan, program akan memutus hubungan antara node target dari daftar anak milik node induknya. Jika node tersebut adalah folder yang berisi item lain, sistem secara otomatis melakukan penghapusan sub-pohon secara rekursif guna mencegah terjadinya kebocoran memori atau adanya node yatim yang tertinggal di dalam sistem.

Berdasarkan pengujian, saat pengguna masuk ke menu 4, program akan memperlihatkan seluruh daftar folder dan file yang berada di lokasi tersebut sebagai panduan bagi pengguna. Begitu pengguna memasukkan kata kunci objek yang ingin dihapus, seperti "job", sistem segera melakukan penelusuran linear, mencocokkan nama, dan menghapus objek node tersebut dari

memori. Setelah struktur Tree diperbarui dan disinkronisasikan ke dalam media penyimpanan permanen, program akan memunculkan notifikasi kepastian berupa “Berhasil menghapus Folder 'job'!” sebelum mengizinkan pengguna menekan tombol Enter untuk kembali menuju menu utama.



```
=====
Pilih menu (0-8): 4

--- HAPUS ITEM ---
Item yang tersedia di lokasi ini:
  [Folder] kerja
  [File] kerjaan
  [Folder] work
  [File] kampus
  [Folder] ily ipb
  [File] kode
  [Folder] job

=====
Kosongkan untuk membatalkan
Masukkan nama yang ingin dihapus: job
Berhasil menghapus Folder 'job' !
Tekan Enter untuk kembali ke menu...
```

```
--- HASIL FILTER ---
[Folder] kerja
[File] kerjaan
[Folder] work
[File] kampus
[Folder] ily ipb
[File] kode
```

3.6 Fitur Searching

Fitur Searching membantu pengguna melacak letak pasti dari suatu berkas atau folder di dalam seluruh sistem repositori. Data direktori diorganisasikan dalam bentuk General Tree sehingga proses pencarian ini menggunakan algoritma penelusuran pohon, yakni Depth-First Search (DFS). Saat kata kunci dimasukkan, algoritma akan melakukan traversal secara rekursif memeriksa nama pada setiap node anak. Jika nama node tersebut cocok atau mengandung fragmen kata kunci yang dicari, sistem akan mencatat *path* lengkap yang ditelusuri dari Root node hingga ke node target tersebut, lalu melanjutkan penelusuran ke cabang pohon lainnya untuk memastikan tidak ada data yang terlewat.

Berdasarkan hasil pengujian, ketika pengguna memilih opsi 7 dan menginput kata kunci nama berupa "ipb", mesin pencari internal mengeksekusi fungsi penelusuran string secara menyeluruh pada semua cabang hierarki. Sistem kemudian berhasil mendeteksi dan menampilkan dua lokasi folder berbeda, yaitu -> [Folder] Laptop_saya/kerja/ipb dan -> [Folder] Laptop_saya/ily ipb. Kemampuan aplikasi untuk melacak dan menyajikan jalur absolut ini membuktikan bahwa algoritma penelusuran objek Tree berjalan secara akurat dan efisien dalam menyisir struktur data bertingkat.

```
=====
Pilih menu (0-8): 7

--- CARI FILE/FOLDER ---
Kosongkan untuk membatalkan
Cari nama: ipb

[Hasil Pencarian untuk 'ipb']:
-> [Folder] Laptop_saya/kerja/ipb
-> [Folder] Laptop_saya/ily ipb

Tekan Enter untuk kembali ke menu... 
```

3.7 Fitur Sorting

Fitur Sorting berfungsi untuk merapikan visualisasi daftar berkas dan folder di dalam lokasi direktori aktif. Sistem akan mengakses daftar anak dari node direktori yang sedang dibuka. Program kemudian menjalankan algoritma pengurutan untuk membandingkan nilai string dari atribut nama pada masing-masing node secara berpasangan. Proses pengurutan ini dilakukan secara in-place pada list kontainer anak sebelum akhirnya dicetak, sehingga tidak merusak hubungan parent-child asli di dalam struktur General Tree.

Berdasarkan hasil eksekusi, ketika pengguna memilih opsi menu 6, sistem langsung memproses seluruh elemen di dalam folder aktif. Layar terminal kemudian menampilkan pesan konfirmasi “Isi folder 'Laptop_saya' berhasil diurutkan A-Z.”, yang segera diikuti dengan pencetakannya. Dapat dilihat bahwa folder dan file yang sebelumnya acak kini telah tersusun rapi secara alfabetis.

```
=====
Pilih menu (0-8): 6

--- URUTKAN A-Z ---

Isi folder 'Laptop_saya' berhasil diurutkan A-Z.

Isi folder setelah diurutkan:
  ily ipb
  kampus
  kerja
  pekerjaan
  kode
  work

Tekan Enter untuk kembali ke menu... 
```

3.8 Penyimpanan Data (File Handling)

Mekanisme penyimpanan data pada aplikasi simulasi ini dirancang menggunakan pendekatan flat-file representation untuk merekonstruksi struktur non-linear General Tree ke dalam media penyimpanan sekunder secara permanen. Format file eksternal yang digunakan adalah CSV (Comma-Separated Values) dengan nama berkas “data_folder.csv.”. Penggunaan format ini dipilih karena sifatnya yang ringan, mudah diurai (parsing), serta representatif dalam memetakan relasi hierarki bertingkat melalui teknik pencatatan referensi induk (parent reference). Setiap baris di dalam file CSV ini merepresentasikan satu buah objek node yang memiliki tiga atribut utama sebagai kolom penunjuk, yaitu Nama (identitas unik berkas/folder), Tipe (pembeda jenis entitas), dan Parent (penunjuk lokasi node induk tempat objek tersebut bernaung).

Saat aplikasi pertama kali dijalankan, fungsi inisialisasi pada modul penyimpanan akan membuka file data_folder.csv secara sekuensial baris demi baris. Sistem membaca data header terlebih dahulu, kemudian melakukan parsing string untuk mengekstrak nilai atribut. Untuk membangun kembali struktur Tree yang utuh di dalam memori, program menggunakan pendekatan iteratif dibantu oleh struktur data Dictionary. Setiap entitas folder yang dibaca akan didaftarkan ke dalam kamus sebagai key, sehingga ketika baris subfolder atau file berikutnya dibaca, program dapat dengan cepat mencari objek parent-nya di dalam kamus dengan kompleksitas waktu konstan $O(1)$, lalu menyisipkannya langsung ke dalam daftar anak (children list) milik node induk tersebut.

Setiap kali pengguna melakukan operasi modifikasi data, baik itu berupa penambahan data (create), pengubahan nama (update), pengurutan (sorting), maupun penghapusan (delete), program akan memicu fungsi tulis-ulang secara menyeluruh demi menjaga konsistensi data. Sistem melakukan traversal atau penelusuran struktur Tree secara menyeluruh mulai dari komponen Root. Selama proses penelusuran tersebut, setiap objek node yang dikunjungi akan diekstrak nilai atributnya, dikonversikan menjadi baris teks berpemisah koma, dan dituliskan kembali secara linear ke dalam file data_folder.csv.

```

data_folder.csv
1  Nama,Tipe,Parent
2  Laptop_saya,Folder,Root
3  kerja,Folder,Laptop_saya
4  file1,File,kerja
5  asd,File,kerja
6  ipb,Folder,kerja
7  ASD,Folder,kerja
8  pekerjaan,File,Laptop_saya
9  work,Folder,Laptop_saya
10 kampus,File,Laptop_saya
11 ily_ipb,Folder,Laptop_saya
12 kode,File,Laptop_saya
13

```

3.9 Validasi Input dan Error Handling

Kesalahan Input	Penanganan
Membuat dua item dengan nama dan tipe yang sama di satu lokasi	Fungsi tambah_isi() akan mendeteksinya lewat perulangan. Sistem menolak perintah tersebut dengan memunculkan pesan error: [Tipe] '[Nama]' sudah ada di lokasi ini!, lalu menunda layar selama 3 detik sebelum otomatis membatalkan proses dan mengembalikan ke menu utama.
Mengubah nama menjadi nama yang sudah dipakai	Pertama, sistem memeriksa apakah item yang mau diubah namanya memang ada. Kedua, sebelum nama diganti, sistem mengecek seluruh isi folder untuk memastikan nama baru tersebut belum dipakai oleh item lain. Jika nama baru itu sudah ada pemiliknya, sistem membatalkan proses dan memunculkan peringatan: Nama '[nama_baru]' sudah digunakan oleh [tipe] lain!.

Membuka file yang dikira folder	Ketika pengguna memilih menu nomor 5 ("Buka Folder") tetapi malah memasukkan nama sebuah file, program mendeteksi atribut tipenya. Karena file tidak bisa dibuka seperti folder, sistem akan mencegah aksi tersebut dan memberi tahu pengguna: Item tersebut adalah File, bukan Folder!, sehingga program tidak rusak.
Pengguna salah menekan nomor menu	Apabila pengguna tidak sengaja mengetik huruf, simbol acak, atau angka di luar pilihan (misalnya mengetik angka 9), sistem mendeteksinya lewat percabangan else paling bawah dan mencetak pesan: Pilihan salah! Harap ketik angka 0-8!.
Berkas penyimpanan CSV belum tersedia atau hilang	Saat program pertama kali dijalankan, modul pembacaan file berada di dalam blok try-except. Apabila berkas data_folder.csv tidak sengaja terhapus atau belum pernah dibuat, sistem tidak akan menampilkan error. Blok except akan langsung mengamankan program dengan memunculkan pesan informatif: "File CSV tidak ditemukan. Memulai dengan folder kosong.", lalu otomatis menyiapkan struktur folder awal secara mandiri.

BAB IV. PENGUJIAN DAN ANALISIS

4.1 Skenario Pengujian

No	Menu	Skenario Pengujian	Hasil
1	Lihat isi folder	Memilih menu 1, menginput angka 3.	Berhasil. Sistem menampilkan seluruh folder dan file di lokasi aktif beserta ikonnya.
2	Tambah folder/file baru	Memilih menu 2, memasukkan nama "job" dan tipe "1" (Folder).	Berhasil. Muncul notifikasi [Sukses] Berhasil menambahkan Folder ke 'Laptop_saya'. Data tersimpan ke CSV.
3	Ubah nama	Memilih menu 3, memasukkan nama folder "job", kemudian mengganti menjadi "Tugas".	Berhasil. Sistem menampilkan pesan "[Sukses] Folder “job” telah diubah menjadi “Tugas.” Perubahan langsung disimpan ke file CSV.
4	Hapus folder/file	Memilih menu 4, memasukkan nama folder "Tugas".	Berhasil. Sistem menampilkan pesan "Berhasil menghapus Folder 'Tugas!'" Data pada file CSV ikut diperbarui.
5	Buka (Folder)	Memilih menu 5, kemudian memasukkan nama folder yang tersedia, misalnya "Dokumen".	Berhasil. Sistem berpindah ke folder “Dokumen”, lokasi aktif berubah sesuai folder yang dipilih dan path saat ini diperbarui.
6	Urutkan (A-Z)	Memilih menu 6 ketika folder aktif memiliki beberapa file/folder dengan nama yang belum berurutan.	Berhasil. Isi folder diurutkan berdasarkan alfabet (A–Z), kemudian sistem menampilkan hasil pengurutan.

7	Cari nama	Memilih menu 7, kemudian memasukkan kata kunci, misalnya "dok".	Berhasil. Sistem melakukan pencarian secara rekursif dan menampilkan seluruh folder/file yang namanya mengandung kata "dok" beserta lokasi (path)-nya.
8	Kembali	Memilih menu 8 setelah masuk ke dalam suatu folder.	Berhasil. Sistem kembali ke folder sebelumnya. Jika sudah berada di folder root, sistem menampilkan pesan bahwa pengguna sudah di lokasi paling awal.
9	Error handling	Memasukkan angka menu di luar pilihan, misalnya angka 9.	Memunculkan peringatan "Pilihan salah! Harap ketik angka 0-8!."

4.2 Kelebihan Sistem

Kelebihan utama dari sistem "Direktori Folder Laptop" ini terletak pada keberhasilan implementasi struktur data non-linear *Tree* (Pohon) yang mampu merepresentasikan arsitektur sistem berkas (*file system*) dunia nyata secara akurat. Dengan memanfaatkan pemrograman berorientasi objek melalui Class Node, sistem memiliki tingkat modularitas yang tinggi, di mana setiap objek folder dan file mengontrol perilakunya sendiri secara mandiri. Hal ini berdampak positif pada efisiensi manajemen memori lokal, karena penambahan atau penghapusan elemen di dalam sub-folder hanya memengaruhi *pointer* referensi pada list internal node yang bersangkutan, tanpa harus merestrukturisasi keseluruhan pohon dari awal.

Sistem ini juga menawarkan fungsionalitas navigasi yang sangat kokoh dan intuitif melalui pemanfaatan struktur data *Stack* (Tumpukan). Implementasi prinsip *Last-In, First-Out* (LIFO) pada runtunan riwayat folder memastikan bahwa pengguna dapat melacak kembali jalur (*backtracking*) ke direktori tingkat atas secara runtut tanpa kehilangan konteks lokasi saat ini.

Selain itu, integrasi algoritma pencarian rekursif (*recursive searching*) memberikan keunggulan dalam hal kapabilitas penelusuran data. Program mampu melakukan pencarian mendalam (*depth-first search*) melewati berbagai lapisan sub-folder yang tidak terbatas jumlahnya, yang mana fitur ini sangat krusial dalam sebuah sistem manajemen berkas modern.

Dari aspek keamanan dan konsistensi data, program telah dilengkapi dengan mekanisme validasi yang komprehensif pada fungsi CRUD. Adanya pengecekan duplikasi nama secara *case-insensitive* serta pembatasan logika (seperti larangan menyisipkan item ke dalam node bertipe "File") berhasil meminimalisasi potensi terjadinya korupsi struktur data akibat kelalaian masukan pengguna (*human error*). Lebih lanjut, ketersediaan fitur otomatisasi sinkronisasi data ke format CSV (*data persistence*) memastikan bahwa setiap perubahan struktur direktori yang dilakukan pengguna akan tersimpan secara permanen, sehingga program dapat memulihkan status data terakhir secara utuh ketika dijalankan kembali di masa mendatang.

4.3 Keterbatasan Sistem

Meskipun sistem Direktori Folder Laptop telah berhasil mengimplementasikan berbagai fitur pengelolaan folder dan file, masih terdapat beberapa keterbatasan yang perlu diperhatikan. Salah satu keterbatasan utama terdapat pada mekanisme penyimpanan data. Setiap kali terjadi proses penambahan, pengubahan, atau penghapusan data, fungsi `simpan_ke_csv()` akan menulis ulang seluruh struktur direktori ke dalam file CSV. Pendekatan ini cukup efektif untuk jumlah data yang kecil, namun ketika jumlah folder dan file semakin banyak, proses penyimpanan akan membutuhkan waktu yang lebih lama karena seluruh node harus ditelusuri dan ditulis kembali. Selain itu, algoritma pencarian yang digunakan juga masih melakukan penelusuran rekursif terhadap seluruh struktur tree sehingga performanya dapat menurun ketika volume data bertambah besar.

Keterbatasan lain ditemukan pada proses pembacaan data dari file CSV saat program dijalankan. Sistem menggunakan nama folder atau file sebagai kunci pada dictionary untuk membangun kembali struktur tree. Akibatnya, apabila terdapat dua folder atau file dengan nama yang sama pada lokasi yang berbeda, proses pemuatan data berpotensi mengalami konflik dan

menyebabkan hubungan parent-child tidak terbentuk dengan benar. Dari sisi fungsionalitas, program juga belum menyediakan fitur lanjutan seperti pemindahan (*move*), penyalinan (*copy*), maupun pengelolaan atribut file yang lebih lengkap seperti ukuran file, tanggal pembuatan, tanggal modifikasi, dan hak akses pengguna sebagaimana yang terdapat pada sistem berkas nyata.

Selain keterbatasan teknis tersebut, sistem masih menggunakan antarmuka berbasis *Command Line Interface* (CLI) yang mengharuskan pengguna berinteraksi melalui menu teks. Meskipun sederhana dan mudah diimplementasikan, pendekatan ini kurang memberikan pengalaman pengguna yang interaktif dibandingkan aplikasi berbasis *Graphical User Interface* (GUI). Program juga belum mendukung penggunaan multi-user, validasi input yang lebih ketat, serta mekanisme pemulihan data apabila terjadi kerusakan pada file CSV. Oleh karena itu, pengembangan lebih lanjut diperlukan untuk meningkatkan efisiensi, keandalan, dan kenyamanan penggunaan sistem sehingga dapat mendekati karakteristik sistem manajemen file yang sebenarnya.

BAB V. KENDALA DAN SOLUSI

Kendala	Solusi
Sensitivitas huruf (case-sensitivity) pada input pengguna	Memanfaatkan fungsi bawaan Python yaitu <code>.lower()</code> pada setiap proses perbandingan string.
Rekonstruksi struktur pohon (tree) dari file CSV yang flat	Menggunakan struktur data dictionary untuk menampung sementara saat membaca file di blok try-except.
Penelusuran folder bertingkat yang terlalu dalam (deep hierarchy)	Menerapkan metode Algoritma Rekursif (fungsi yang memanggil dirinya sendiri) pada fungsi <code>_tulis_rekursif()</code> untuk penyimpanan data dan <code>_cari_rekursif()</code> untuk pencarian data.
Melakukan pencarian kata kunci parsial	Di dalam fungsi <code>_cari_rekursif()</code> , mengganti operator perbandingan sama dengan (<code>==</code>) menjadi operator keanggotaan in (yaitu: <code>if nama_cari.lower() in node.folder.lower():</code>).

Pengalaman paling menantang bagi kelompok kami adalah menyatukan logika kerja ketiga anggota ke dalam satu file program yang utuh. Kami harus memastikan bahwa setiap kali ada perubahan data (seperti setelah menu tambah, ubah nama, urutkan, atau hapus), fungsi `simpan_ke_csv()` wajib langsung dipanggil agar data di laptop pengguna selalu sinkron dan tidak ada data yang hilang saat program ditutup.

BAB VI. PEMBAGIAN TUGAS ANGGOTA

Nama	Tugas
Mouzia Syawalova Merro	- Merancang struktur dasar program - Mengimplementasikan logika utama operasi CRUD
Alyaa Mardlatillah Sofwan	- Mengembangkan algoritma pemrosesan data, pengurutan, pencarian - Manajemen berkas eksternal
Gabriella Anggi Apriliana Tambun	- Membangun alur program utama - Tampilan antarmuka CLI - Error handling

BAB VII. KESIMPULAN DAN SARAN

Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan terhadap Aplikasi Simulasi Manajemen Direktori Folder Laptop ini, dapat ditarik beberapa kesimpulan utama sebagai berikut:

1. Aplikasi simulasi manajemen direktori folder laptop berhasil dibangun berbasis CLI menggunakan Python dengan mengimplementasikan seluruh operasi CRUD secara aman.
2. Struktur data *General Tree* sukses digunakan untuk mewakili hierarki berkas bertingkat, sementara struktur data *Stack* terbukti efektif dalam mengelola riwayat navigasi maju-mundur folder.
3. Konsep *file handling* menggunakan format CSV berhasil diterapkan untuk menjaga persistensi data, sehingga seluruh struktur folder tersimpan permanen dan tidak hilang saat program ditutup.

Saran Pengembangan

Demi penyempurnaan sistem simulasi ini di masa yang akan datang, beberapa fitur potensial yang direkomendasikan untuk dikembangkan adalah:

1. Mengembangkan tampilan antarmuka aplikasi dari berbasis teks (CLI) menjadi berbasis grafis (GUI) agar lebih interaktif dan mudah digunakan oleh pengguna awam.
2. Menambahkan fitur manajemen ukuran berkas serta pembatasan kapasitas penyimpanan pada masing-masing folder.

DAFTAR PUSTAKA

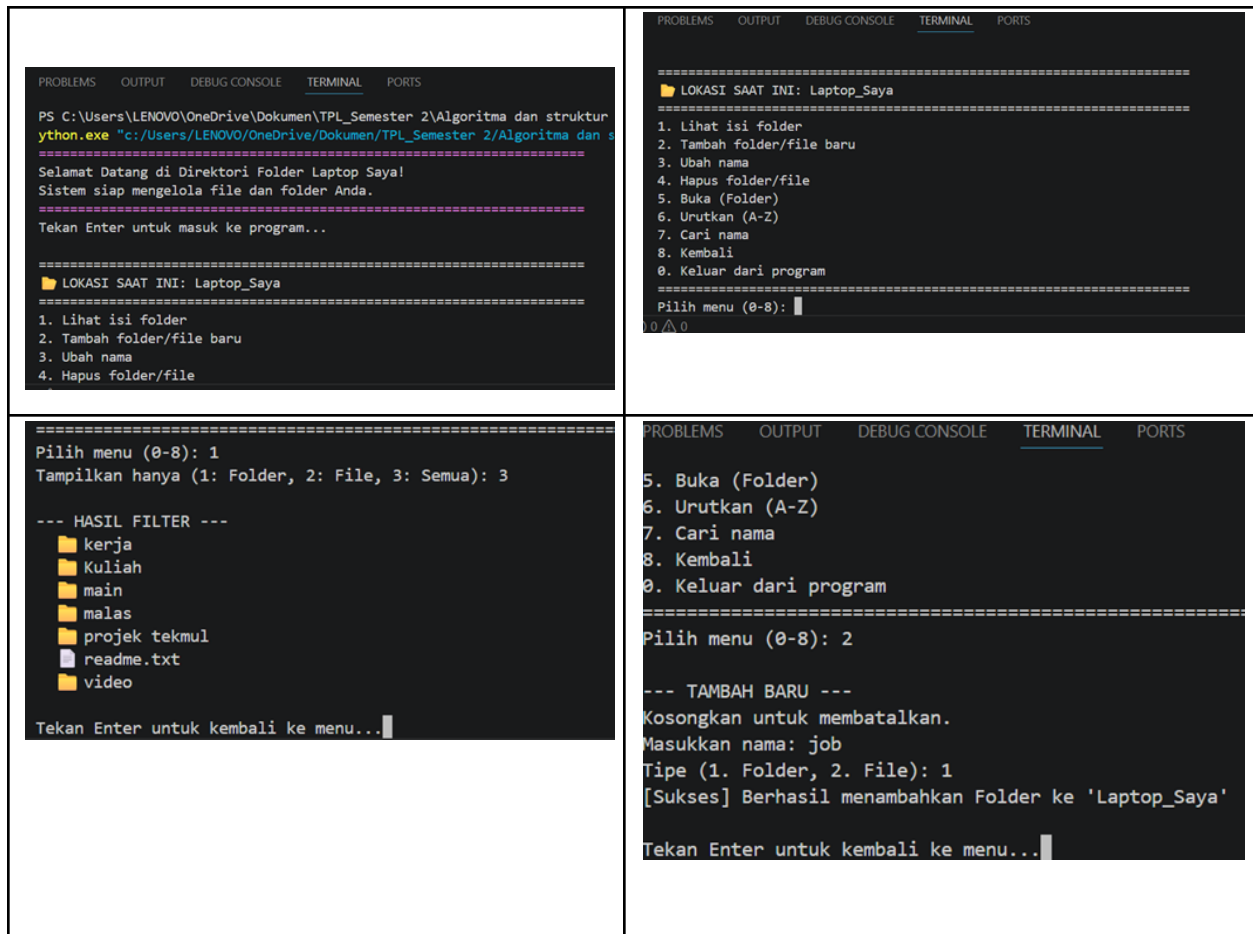
- Setiawan, A., & Budiman, A. (2024). Implementasi Struktur Data Tree pada Sistem Manajemen Berkas Berbasis Perintah Suara. *Journal of Applied Science, Information Systems and Computing (JASIC)*, 5(1), 35-42. <https://journal.unesa.ac.id/index.php/jasic/article/view/44459>
- Nurkamid, M., & Khotimah, T. (2023). Analisis Efisiensi Penyimpanan Data Bertingkat Menggunakan Flat-File Representation pada File CSV. *MIFORTEKH: Jurnal Mikroskil, Formasi, Teknologi dan Kesehatan*, 2(2), 112-120. <https://journal.stiestekom.ac.id/index.php/mifortekh/article/view/742>
- Zhao, X. (2022). Basic search algorithms in Python: Review on linear and binary implementations. *Proceedings of SPIE*, 12287, Article 1228704. <https://doi.org/10.1117/12.2640473>
- Wang, Y., & Liu, J. (2022). Practical implementation of tree structures and traversal algorithms in dynamic memory systems. Dalam *Advances in Computer Science and Educational Technology* (pp. 67-78). Springer, Cham. https://doi.org/10.1007/978-3-032-01442-9_6

LAMPIRAN

Lampiran A. Link GitHub

https://github.com/ayayiee/ASD_Kelompok_18.git

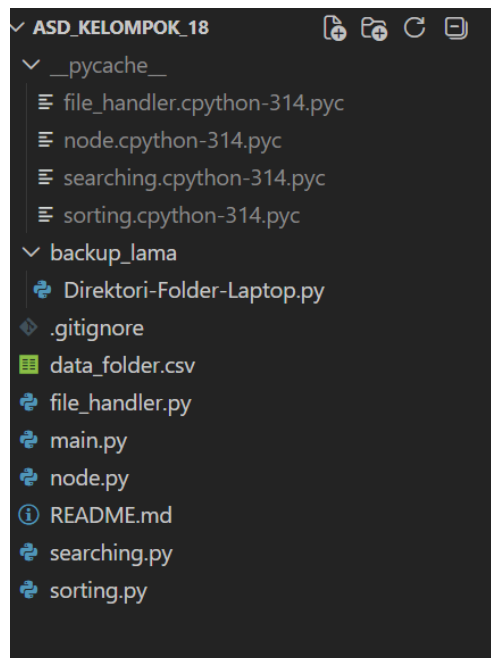
Lampiran B. Screenshot Program



<pre> PROBLEMS OUTPUT DEBUG CONSOLE <u>TERMINAL</u> PORTS --- UBAH NAMA --- Item yang tersedia di lokasi ini: [Folder] kerja [Folder] Kuliah [Folder] main [Folder] malas [Folder] projek tekmul [File] readme.txt [Folder] video [Folder] job ----- (Kosongkan untuk membatalkan) Masukkan nama yang ingin diubah: job Masukkan nama baru: tugas </pre>	<pre> --- HAPUS ITEM --- Item yang tersedia di lokasi ini: [Folder] kerja [Folder] Kuliah [Folder] main [Folder] malas [Folder] projek tekmul [File] readme.txt [Folder] video [Folder] tugas ===== Kosongkan untuk membatalkan Masukkan nama yang ingin dihapus: tugas Berhasil menghapus Folder 'tugas'! ! </pre>
<pre> Folder yang tersedia : [Folder] kerja [Folder] Kuliah [Folder] main [Folder] malas [Folder] projek tekmul [Folder] video [Folder] dokumen ===== Kosongkan untuk membatalkan Masukkan nama folder untuk dibuka: dokumen [Sistem] Berhasil masuk ke folder 'dokumen' </pre>	<pre> ===== [Folder] LOKASI SAAT INI: Laptop_Saya / dokumen ===== 1. Lihat isi folder 2. Tambah folder/file baru 3. Ubah nama 4. Hapus folder/file 5. Buka (Folder) 6. Urutkan (A-Z) 7. Cari nama 8. Kembali 0. Keluar dari program ===== Pilih menu (0-8): </pre>
<pre> --- URUTKAN A-Z --- Isi folder 'Laptop_Saya' berhasil diurutkan A-Z. Isi folder setelah diurutkan: [Folder] dokumen [Folder] kerja [Folder] Kuliah [Folder] main [Folder] malas [Folder] projek tekmul [File] readme.txt [Folder] video Tekan Enter untuk kembali ke menu... </pre>	<pre> 6. Urutkan (A-Z) 7. Cari nama 8. Kembali 0. Keluar dari program ===== Pilih menu (0-8): 7 --- CARI FILE/FOLDER --- Kosongkan untuk membatalkan Cari nama: dok [Hasil Pencarian untuk 'dok']: -> [Folder] Laptop_Saya/dokumen Tekan Enter untuk kembali ke menu... </pre>

<pre> 6. Urutkan (A-Z) 7. Cari nama 8. Kembali 0. Keluar dari program ===== Pilih menu (0-8): 8 >>> Kembali ke folder sebelumnya... ===== 📁 LOKASI SAAT INI: Laptop_Saya ===== 1. Lihat isi folder 2. Tambah folder/file baru 3. Ubah nama 🚩 0 </pre>	<pre> 7. Cari nama 8. Kembali 0. Keluar dari program ===== Pilih menu (0-8): 9 Pilihan salah! Harap ketik angka 0-8! ===== 📁 LOKASI SAAT INI: Laptop_Saya ===== 1. Lihat isi folder 2. Tambah folder/file baru 3. Ubah nama 4. Hapus folder/file 🚩 0 </pre>
--	---

Lampiran C. Struktur Folder Proyek



Lampiran D. Commit History GitHub

Commits on Apr 20, 2026	Delete xml.jsp2bec406 Add files via upload4bfc838
Commits on Apr 15, 2026	Add files via uploada9c8e86
Commits on Apr 12, 2026	fix conflict a1acc7b first commit cbc9876 Pengerjaan awal oleh Mouzia Syawalova Merro eac56fc
Commits on May 19, 2026	Rename crud fin.py to Direktori Laptop.py e0ead90 Direktori Laptop.py 4803808 Menambahkan penutup di menu keluar 5d8c29d
Commits on May 12, 2026	Update crud fin.py c1ea44c Update crud fin.py 77b71d8 Update crud fin.py eea426b Update crud fin.py faeacc6
	perbaikan fitur direktori dan optimasi jeda time sleep bb149a0
Commits on May 20, 2026	FINALISASI SELURUH KODE f2e0cdc Update README kesalahan penamaan.md eb337b9 Update README tanpa input x.md ec181fd Update README.md e6a26b6 Update README.md e82cc33 Create README.md 8731e84 Merapikan seluruh main menu dan finalisasi kode 7440449
Commits on Jun 26, 2026	Refactor: Memecah kode menjadi struktur modular dan merapikan direktori proyek 9fa2e44
Commits on Jun 7, 2026	perbaikan fitur direktori dan optimasi jeda time sleep bb149a0

Lampiran E. Source Code Utama

```
node.py X
node.py > % Node
1 import time
2
3 pink = '\033[95m'
4 reset = '\033[0m'
5
6 # === TUGAS ANGOTA 1: MUZIIA SYAHALOVA ===
7 class Node:
8     def __init__(self, folder, tipe):
9         self.folder = folder
10        self.tipe = tipe
11        self.isi = []
12        self.posisi = None
13
14        # =====
15        # CREATE
16        # =====
17        def tambah_isi(self, node_baru):
18            if self.tipe == "File":
19                print("Tidak bisa menambahkan item ke dalam File!")
20                return False
21            for item in self.isi:
22                if item.folder.lower() == node_baru.folder.lower() and item.tipe == node_baru.tipe:
23                    print(f"({node_baru.tipe}) '{node_baru.folder}' sudah ada di lokasi ini!")
24                    time.sleep(3)
25                    return False
26
27            node_baru.posisi = self
28            self.isi.append(node_baru)
29            return True
```

```
node.py X
node.py > % Node
7 class Node:
8     # =====
9     # READ
10    # =====
11    def lihat_isi(self, filter_tipe=None):
12        items = self.isi
13        if filter_tipe:
14            items = [i for i in self.isi if i.tipe == filter_tipe]
15            print(f" (Menampilkan hanya: {filter_tipe})")
16
17        if not items:
18            print("Folder Kosong - Gunakan menu 'Tambah' untuk mengisi folder ini.")
19        else:
20            for item in items:
21                ikon = "📁" if item.tipe == "Folder" else "📄"
22                print(f" {ikon} {item.folder}")
23
24    # =====
25    # UPDATE
26    # =====
27    def ubah_nama(self, nama_lama, nama_baru):
28        for item in self.isi:
29            if item.folder.lower() == nama_lama.lower():
30                for cek in self.isi:
31                    if cek.folder.lower() == nama_baru.lower() and cek.tipe == item.tipe:
32                        print(f"Nama '{nama_baru}' sudah digunakan oleh {item.tipe} lain!")
33                        time.sleep(2)
34                        return False
```

```
node.py X
node.py > % Node
7 class Node:
8     # =====
9     # DELETE
10    # =====
11    def hapus_isi(self, nama_dihapus):
12        for item in self.isi:
13            if item.folder.lower() == nama_dihapus.lower(): #lower membuat jadi huruf kecil agar tidak sensitif
14                self.isi.remove(item)
15                print(f"Berhasil menghapus {item.tipe} '{item.folder}'!")
16                return True
17            print(f"Gagal, {nama_dihapus} tidak ditemukan.")
18            return False
19
20    # =====
21    # DELETE
22    # =====
23    def ubah_nama(self, nama_lama, nama_baru):
24        for item in self.isi:
25            if item.folder.lower() == nama_lama.lower():
26                if cek.folder.lower() == nama_baru.lower() and cek.tipe == item.tipe:
27                    print(f"Nama '{nama_baru}' sudah digunakan oleh {item.tipe} lain!")
28                    time.sleep(2)
29                    return False
30                item.folder = nama_baru
31                print(f"Sukses! {item.tipe} '{nama_lama}' telah diubah menjadi '{nama_baru}'.")
32                return True
33            print(f"Gagal, {nama_lama} tidak ditemukan di folder ini.")
34            return False
35
36    # =====
37    # DELETE
38    # =====
39    def hapus_isi(self, nama_dihapus):
40        for item in self.isi:
41            if item.folder.lower() == nama_dihapus.lower(): #lower membuat jadi huruf kecil agar tidak sensitif
42                self.isi.remove(item)
43                print(f"Berhasil menghapus {item.tipe} '{item.folder}'!")
44                return True
45            print(f"Gagal, {nama_dihapus} tidak ditemukan.")
46            return False
```

```
node.py X file_handler.py X
file_handler.py > ...
1 import csv
2 from node import Node
3
4 # === TUGAS ANGOTA 2: ALYAA HARRDLATIL LAM ===
5 # =====
6 # Menyimpan struktur folder ke CSV
7 # =====
8 def simpan_ke_csv(node_root, nama_file="data_folder.csv"):
9     try:
10         with open(nama_file, mode='w', newline='', encoding='utf-8') as f:
11             writer = csv.writer(f)
12             writer.writerow(["Nama", "Tipe", "Parent"]) # Header
13             writer.writerow([node_root.folder, node_root.tipe, "Root"])
14             _tulis_rekursif(node_root, writer)
15     except Exception as e:
16         print(f"[Error CSV] {e}")
17
18 # =====
19 # Mencatat setiap folder dan file ke dalam baris CSV
20 # =====
21 def _tulis_rekursif(node, writer):
22     """Mencatat setiap folder dan file ke dalam baris CSV"""
23     for item in node.isi:
24         writer.writerow([item.folder, item.tipe, node.folder])
25         if item.tipe == "Folder":
26             _tulis_rekursif(item, writer)
```

```
sorting.py X
sorting.py > ...
1 # =====
2 # Sorting (mengurutkan isi folder A-Z)
3 # =====
4 def urutkan_isi(node_sekarang):
5     if node_sekarang.isi:
6         node_sekarang.isi.sort(key=lambda x: x.folder.lower())
7         print(f"\n Isi folder '{node_sekarang.folder}' berhasil diurutkan A-Z.")
8     else:
9         print("\n Folder kosong, tidak ada yang bisa diurutkan.")
10
```

```
searching.py X
searching.py > ...
1 # =====
2 # Searching (mencari file/folder di seluruh sistem)
3 # =====
4 def cari_file_folder(node_root, nama_cari):
5     hasil = []
6     _cari_rekursif(node_root, nama_cari, hasil)
7
8     if hasil:
9         print(f"\nHasil Pencarian untuk '{nama_cari}':")
10        for path in hasil:
11            print(f"-> {path}")
12    else:
13        print(f"\n'{nama_cari}' tidak ditemukan.")
14
15 # =====
16 # Searching (mencari ke dalam semua sub-folder)
17 # =====
18 def _cari_rekursif(node, nama_cari, hasil, path_sekarang=""):
19     path_baru = f"{path_sekarang}/{node.folder}" if path_sekarang else node.folder
20     if nama_cari.lower() in node.folder.lower():
21         hasil.append(f"[{node.tipe}] {path_baru}")
22
23     for item in node.isi:
24         _cari_rekursif(item, nama_cari, hasil, path_baru)
```

main.py X

```
main.py > ...
1 import csv
2 import time
3 from node import Node
4 from file_handler import simpan_ke_csv
5 from sorting import urutkan_isi
6 from searching import cari_file_folder
7
8 pink = '\033[95m'
9 reset = '\033[0m'
10
11 # == TUGAS ANGKOTA 3: GABRIELLA ANGGI ==
12 root = Node("Laptop_saya", "Folder")
13
14 print(f"{pink}" + "*"70 + f"{reset}")
15 print("Selamat Datang di Direktori Folder Laptop Saya!")
16 print("Sistem siap mengelola file dan folder Anda.")
17 print(f"{pink}" + "*"70 + f"{reset}")
18 input("Tekan Enter untuk masuk ke program...")
19
20 try:
21     nodes = {}
22     with open("data_folder.csv", mode='r', encoding='utf-8') as f:
23         reader = csv.reader(f)
24         next(reader)
25         for baris in reader:
26             nama, tipe, parent = baris
27             baru = Node(nama, tipe)
28             nodes[nama] = baru
29             if parent == "Root":
```

main.py X

```
main.py > ...
30         root = baru
31     else:
32         if parent in nodes:
33             nodes[parent].tambah_isi(baru)
34 except:
35     print("File CSV tidak ditemukan. Memulai dengan folder kosong.")
36
37 node_sekarang = root
38 stack_riwayat = []
39
40 while True:
41     path_saatini = " / ".join([n.folder for n in stack_riwayat] + [node_sekarang.folder])
42     print("\n" + "*"70)
43     print(f"LOKASI SAAT INI: {path_saatini}")
44     print("="70)
45     print("1. Lihat isi folder")
46     print("2. Tambah folder/file baru")
47     print("3. Ubah nama")
48     print("4. Hapus folder/file")
49     print("5. Buka (folder)")
50     print("6. Urutkan (A-Z)")
51     print("7. Cari nama")
52     print("8. Kembali")
53     print("9. Keluar dari program")
54     print("="70)
55
56 pilihan = input("Pilih menu (0-8): ").strip()
57
58 if pilihan == "1":
```

main.py X

```
main.py > ...
58 if pilihan == "1":
59     f = input("Tampilkan hanya (1: Folder, 2: File, 3: Semua): ")
60     if f in ["1", "2", "3"]:
61         tipe_f = "Folder" if f=="1" else "File" if f=="2" else None
62         print("\n--- HASIL FILTER ---")
63         node_sekarang.lihat_isi(tipe_f)
64     else:
65         print("\nPilihan tidak tersedia!")
66     input("\nTekan Enter untuk kembali ke menu...")
67
68 elif pilihan == "2":
69     print("\n--- TAMBAH BARU ---")
70     print("Kosongkan untuk membatalkan.")
71
72     nama_baru = input("Masukkan nama: ").strip()
73     if not nama_baru:
74         print("\n>>> Penambahan item dibatalkan. Kembali ke menu utama...")
75         time.sleep(3)
76         continue
77
78     tipe_input = input("Tipe (1. Folder, 2. File): ").strip()
79     if not tipe_input:
80         print("\n>>> Penambahan item dibatalkan. Kembali ke menu utama...")
81         time.sleep(3)
82         continue
83
84     tipe_baru = "Folder" if tipe_input == "1" else "File"
85     if node_sekarang.tambah_isi(Node(nama_baru, tipe_baru)):
86         print(f"[Sukses] Berhasil menambahkan {tipe_baru} ke '{node_sekarang.folder}'")
```

main.py X

```
main.py > ...
86         print(f"[Sukses] Berhasil menambahkan {tipe_baru} ke '{node_sekarang.folder}'")
87         simpan_ke_csv(root)
88         input("\nTekan Enter untuk kembali ke menu...")
89
90 elif pilihan == "3":
91     print("\n--- UBAH NAMA ---")
92
93     if not node_sekarang.isi:
94         print("Folder kosong, tidak ada item yang bisa diubah namanya")
95         input("\nTekan Enter untuk kembali ke menu...")
96         continue
97
98     print("Item yang tersedia di lokasi ini:")
99     for item in node_sekarang.isi:
100         ikon = "📁" if item.tipe == "Folder" else "📄"
101         print(f"{ikon} {item.folder} ({item.tipe})")
102     print("\n" + "*"70)
103     print("Kosongkan untuk membatalkan")
104
105     nama_lama = input("Masukkan nama yang ingin diubah: ")
106     if not nama_lama:
107         print("\n>>> Perubahan nama dibatalkan. Kembali ke menu utama...")
108         time.sleep(3)
109         continue
110
111     nama_ketemu = False
112     for item in node_sekarang.isi:
113         if item.folder.lower() == nama_lama.lower():
```

main.py X

```
main.py > ...
115         if not break:
116             if not nama_ketemu:
117                 print(f"⚠️Gagal! '{nama_lama}' tidak ditemukan di folder ini.")
118                 input("\nTekan Enter untuk kembali ke menu...")
119                 continue
120
121             nama_baru = input("Masukkan nama baru: ")
122             if not nama_baru:
123                 print("\n>>> Proses penghapusan dibatalkan. Kembali ke menu utama...")
124                 time.sleep(3)
125                 continue
126
127             if node_sekarang.ubah_nama(nama_lama, nama_baru):
128                 simpan_ke_csv(root)
129                 input("\nTekan Enter untuk kembali ke menu...")
130
131 elif pilihan == "4":
132     print("\n--- HAPUS ITEM ---")
133     if not node_sekarang.isi:
134         print("\nFolder kosong, tidak ada file atau folder yang bisa dihapus.")
135         input("Tekan Enter untuk kembali ke menu...")
136         continue
137
138     print("Item yang tersedia di lokasi ini:")
139     for item in node_sekarang.isi:
140         ikon = "📁" if item.tipe == "Folder" else "📄"
141         print(f"{ikon} {item.folder} ({item.tipe})")
142     print("\n" + "*"70)
143     print("Kosongkan untuk membatalkan")
```

main.py X

```
main.py > ...
145     nama_hapus = input("Masukkan nama yang ingin dihapus: ").strip()
146
147     if not nama_hapus:
148         print("\n>>> Proses penghapusan dibatalkan. Kembali ke menu utama...")
149         time.sleep(3)
150         continue
151
152     if node_sekarang.hapus_isi(nama_hapus):
153         simpan_ke_csv(root)
154         input("\nTekan Enter untuk kembali ke menu...")
155
156 elif pilihan == "5":
157     print("\n--- BUKA FOLDER ---")
158     daftar_folder = [item for item in node_sekarang.isi if item.tipe == "Folder"]
159
160     if not daftar_folder:
161         print("Tidak ada folder yang bisa dibuka di lokasi ini.")
162         input("Tekan Enter untuk kembali ke menu...")
163         continue
164
165     else:
166         print("\nFolder yang tersedia:")
167         for folder_aktif in daftar_folder:
168             print(f"📁 {folder_aktif.folder}")
169         print("\n" + "*"70)
170
171     while True:
172         print("\nKosongkan untuk membatalkan")
173         target = input("Masukkan nama folder untuk dibuka: ").strip()
```

```

main.py x
main.py > ...
175         if not target:
176             print("\n>>> Proses membuka folder dibatalkan. Kembali ke menu utama...")
177             time.sleep(2)
178             break
179
180         found = False
181         for item in node_sekarang.isi:
182             if item.folder.lower() == target.lower():
183                 found = True
184                 if item.tipe == "Folder":
185                     stack_riwayat.append(node_sekarang)
186                     node_sekarang = item
187                     print(f"[Sistem] Berhasil masuk ke folder '{node_sekarang.folder}'")
188                     time.sleep(2)
189                 else:
190                     print("Item tersebut adalah File, bukan Folder!")
191                     found = False; break
192             if found:
193                 break
194         elif target.lower() != 'x':
195             print("Folder tidak ditemukan. Silakan periksa ejaan dan coba lagi.")
196             print("\n"*70)
197
198         elif pilihan == "6":
199             print("\n--- URUTKAN A-Z ---")
200             urutkan_isi(node_sekarang)
201
202             if node_sekarang.isi:
203                 print("\nIsi folder setelah diurutkan:")

```

```

main.py •
main.py > ...
204         node_sekarang.lihat_isi()
205         input("\nTekan Enter untuk kembali ke menu...")
206
207         elif pilihan == "7":
208             print("\n--- CARI FILE/FOLDER ---")
209             print("Kosongkan untuk membatalkan")
210             cari_file_folder(root, input("Cari nama: "))
211             input("\nTekan Enter untuk kembali ke menu...")
212
213         elif pilihan == "8":
214             if stack_riwayat:
215                 print("\n>>> Kembali ke folder sebelumnya...")
216                 time.sleep(2)
217                 node_sekarang = stack_riwayat.pop()
218             else:
219                 print("\nAnda sudah berada di lokasi paling awal (Root).")
220                 print("Kembali ke menu utama dalam 2 detik ... ")
221                 time.sleep(2)
222
223         elif pilihan == "0":
224             simpan_ke_csv(root)
225             print(f"\n{pink}" + "="*70 + f"{reset}")
226             print("Program ditutup.")
227             print("Terimakasih telah menggunakan Direktori Folder Laptop!")
228             print(f"{pink}" + "="*70 + f"{reset}")
229             break
230
231         else:
232             print("\nPilihan salah! Harap ketik angka 0-8!")

```